Interrogation 1 (12 octobre)

durée : 1 heure

Pour chacun des algorithmes, on justifiera avec soin :

- que l'algorithme termine.
- que l'algorithme renvoie bien le résultat demandé.

Pour les boucles « pour », on adoptera les conventions suivantes. Si $a, b \in \mathbb{Z}$, « pour i allant de a à b » signifie « pour i parcourant en croissant l'intervalle $[\![a, b]\!]$ ». Lorsque $b < a$, cet intervalle est vide, donc aucune des instructions dans la boucle « pour » n'est effectuée (ce sera par exemple le cas si on écrit « pour i allant de 1 à n », avec $n = 0$). On peut aussi écrire les algorithmes en Python, dans ce cas le code Python doit être correct, en particulier l'utilisation de « range » doit être correcte. On rappelle qu'en Python, les listes sont numérotées à partir de zéro, `len(L)` retourne la longueur d'une liste, et que `L[a:b]` retourne la liste extraite en prenant les indices entre a inclus et b exclus. L'expression `L[a:]` retourne la liste (éventuellement vide) extraite en prenant les indices à partir de a. La concaténation de deux listes se fait avec l'opérateur `+`.

(Rappel : en cas de blocage, la méthodologie attendue est d'abord de faire de nombreux exemples.)

Exercice 1. *On considère la fonction suivante, qui prend deux entiers relatifs en entrée. Termine-t-elle ? Que fait-elle ?*

```
def f(n,k):
  if k < 0 or k > n:
    return 0
  if k == 0 or n == k:
    return 1
  return f(n-1,k)+f(n-1,k-1)
```

Exercice 2. *On désire écrire une fonction prenant en entrée un nombre entier positif et renvoyant la liste des chiffres de son écriture en base deux, dans l'ordre d'écriture des chiffres. Pour l'entrée* 6*, la fonction doit renvoyer* `[1,1,0]`*.*

1. *Écrire une version itérative de la fonction. (La fonction ne doit pas utiliser la récursivité, uniquement des boucles.)*
2. *Écrire une version récursive. (Aucune boucle.)*

Exercice 3. *L'algorithme suivant prend un réel x et un entier positif n. Il est implémenté en Python, avec un flottant et un entier, qui est toujours supposé positif. Montrer qu'elle termine et retourne x^n.*

```
def f(x:float, n: int)-> float:
  r = 1
  p = x
  while n>0:
    if n % 2 == 1:
      r = r * p
    p = p * p
    n = n // 2
  return r
```

Exercice 4. *Réécrire la fonction* `f` *de l'exercice précédent sans boucle while, en utilisant la récursivité, mais en suivant la même idée.*

Proposition de corrigé

Rappel de cours sur la division euclidienne dans $\mathbb{N}$: soit $a \in N$ et $b \in \mathbb{N}^*$. Il existe un unique couple $(r, q) \in \mathbb{N} \times [\![0, b-1]\!]$ tel que $a = bq + r$. L'entier q est le quotient entier de la division de a par b, et dans ce document on le notera $a//b$. Cette notation n'est pas standard en maths, si vous l'utilisez dans une copie vous devez la redéfinir. L'entier r est le reste de la division euclidienne de a par b et on notera $a\%b$. Même remarque.

Attention, $a\%b$ est un entier, pas une classe de congruence. C'est l'unique élément dans la classe de congruence de a modulo b qui est compris entre 0 et $b-1$. Il est licite d'écrire qu'il est égal à un autre entier, ou inférieur etc. Ne pas confondre avec la notation $a \mod b$ en maths qui nécessite l'emploi de congruences.

Remarque. *On a $q = \lfloor a/b \rfloor$ mais utiliser cette forme complique souvent la rédaction, en général il vaut mieux écrire ce que donne la division euclidienne et laisser les parties entières au cas où vraiment on ne peut pas faire sans.*

Remarque. *Si c est un autre naturel non nul, alors :*

$$a//(bc) = (a//b)//c.$$

En effet, écrivons les deux divisions euclidiennes :

$$a = b \times (a//b) + r, \quad a//b = c \times ((a//b)//c) + r',$$

avec $r = a\%b \in [\![0, b-1]\!]$ et $r' = (a//b)\%c \in [\![0, c-1]\!]$. En remplaçant, on obtient :

$$(E) \qquad a = bc \times ((a//b)//c) + br' + r.$$

Comme $r' \in [\![0, c-1]\!]$, on a $br' \in [\![0, b(c-1)]\!]$. En ajoutant $r \in [\![0, b-1]\!]$, on obtient $br' + r \in [\![0, b(c-1) + (b-1)]\!] = [\![0, bc-1]\!]$.

L'égalité (E) plus haut est donc la division euclidienne de a par bc et par unicité de cette division euclidienne de a par bc, on a donc $(a//b)//c = a//(bc)$.

Si ce raisonnement est utilisé dans une copie de capes, il faut le rédiger en totalité comme ci-dessus. Il est utilisé plusieurs fois dans la correction qui suit.

Exercice 1

(Remarque préliminaire : il y a beaucoup de façons de rédiger correctement la preuve. Il faut faire rapidement un choix sur les définitions et sur la façon — et le moment – de gérer les indices négatifs ou $k > n$.)

Dans cette correction, on suppose définis les coefficients binomiaux $\binom{n}{k}$ uniquement pour $n \in \mathbb{N}$ et $0 \leq k \leq n$. C'est un cadre restrictif (on pourrait les définir sur un domaine plus grand de $\mathbb{Z}^2$), mais qui suffira. Rappelons la formule de Pascal dans ce contexte :

$$\forall n \geq 1, \forall k \in [\![1, n]\!], \binom{n+1}{k} = \binom{n}{k} + \binom{n}{k-1}.$$

Passons à l'algorithme. Montrons que la fonction termine, et qu'elle renvoie zéro si $k < 0$ ou $k > n$, et sinon (autrement dit si $0 \leq k \leq n$) qu'elle renvoie le coefficient binomial $\binom{n}{k}$.

Pour $k < 0$ ou $k > n$, ça renvoie immédiatement zéro, ce qu'il fallait démontrer.

Montrons maintenant que la fonction termine et renvoie la valeur annoncée pour des valeurs $0 \leq k \leq n$. On le fait par récurrence sur n.

Pour tout entier naturel n, on note $\mathcal{A}(n)$ l'assertion : « pour tout k tel que $0 \leq k \leq n$, la fonction $f(n, k)$ termine et renvoie $\binom{n}{k}$ ». Montrons par récurrence : $\forall n \in \mathbb{N}, \mathcal{A}(n)$. Dans la suite, « renvoie » veut dire « termine et renvoie », pour alléger la rédaction.

1. **Initialisation**. Montrons $\mathcal{A}(0)$ ainsi que $\mathcal{A}(1)$. L'assertion $\mathcal{A}(0)$ est équivalente à : $f(0,0) = \binom{0}{0} = 1$. Or, pour des valeurs d'entrée $n = 0$ et $k = 0$, la fonction saute le premier « if », puis elle rentre dans le second « if » car $k = 0$ et renvoie bien 1, d'où l'assertion $\mathcal{A}(0)$. On vérifie également que $\mathcal{A}(1)$ est vraie.
2. **Hérédité**. Soit $n \geq 1$. Supposons $\mathcal{A}(n)$. Montrons $\mathcal{A}(n+1)$. Soit $k \in \mathbb{N}$ avec $0 \leq k \leq n+1$. Il y a trois possibilités :
   (a) Si $k = 0$, la fonction entre dans le second « if », et renvoie $1 = \binom{n+1}{0} = \binom{n+1}{k}$, ce qu'il fallait démontrer.
   (b) Si $k = n+1$, alors la fonction entre aussi dans le second « if » et renvoie $1 = \binom{n+1}{n+1} = \binom{n+1}{k}$, ce qu'il fallait démontrer.
   (c) Sinon, on a donc $1 \leq k \leq n$. On ne rentre dans aucun des deux « if ». La fonction appelle alors $f((n+1)-1,k)$ et $f((n+1)-1,k-1)$, c'est-à-dire $f(n,k)$ et $f(n,k-1)$. Comme $1 \leq k \leq n$, on a $0 \leq k \leq n$ et $0 \leq k-1 \leq n$. D'après l'hypothèse de récurrence, $f(n,k)$ renvoie $\binom{n}{k}$ et $f(n,k-1)$ renvoie $\binom{n}{k-1}$. La fonction renvoie donc $\binom{n}{k-1} + \binom{n}{k} = \binom{n+1}{k}$ par la formule de Pascal, ce qu'il fallait démontrer.

Note : on peut prouver la terminaison de façon autonome au début, en séparant terminaison et correction. La fonction termine si $n < 0$, et sinon, on prouve la terminaison par récurrence sur $n \in \mathbb{N}$. Mais ceci rallonge inutilement la preuve, on se retrouve à écrire souvent deux fois la même chose.

Erreurs vues : l'assertion $\mathcal{A}(n)$ doit dépendre de n (donc ne pas commencer par $\forall n$!). S'il y a d'autres symboles à l'intérieur de cette assertion, ils doivent être quantifiés. Ou alors, il peut s'agir d'objets déjà définis (dans l'énoncé, ou dans la copie, voir par exemple plus bas avec le réel x qui est fixé dès le début).

Conseils : avant de tenter des choses très compliquées (récurrences sur $n+k$, autres récurrences exotiques, ordres lexicographiques etc), en présence de plusieurs indices, tenter simplement une récurrence sur l'un ou l'autre, en quantifiant correctement le second indice.

Conseils de rédaction : cet exo est en fait un exo de récurrence, qui teste aussi si on est minutieux sur les cas nécessitant une disjonction (négatifs, ou $k > n$). Des points sont enlevés si l'assertion $\mathcal{A}(n)$ est incorrecte, si on confond entre récurrence forte ou simple, et si on n'explique pas très clairement comment et à qui on applique l'hypothèse de récurrence (et la raison pour laquelle on a le droit de le faire). Ensuite dans ce cas particulier, il faut vraiment citer correctement et explicitement la formule de Pascal, pour les bons indices etc.

Exercice 2 : écriture en base deux

Algorithme itératif

```
def binaire(n: int)-> list:
  if n == 0 : return [0]
  L = []
  while n>0:
    L = [n % 2] + L
    n = n // 2
  return L
```

Preuve de terminaison : Tant que la boucle tourne, c'est-à-dire tant que $n > 0$, la suite des valeurs de n est une suite strictement décroissante d'entiers positifs, puisque si $n > 0$, alors $n//2 < n$. Cette suite est donc finie, autrement dit la boucle est finie. On note $K \in \mathbb{N}$ le nombre de tours de boucle effectués (qui peut être nul si $n = 0$).

Preuve de correction :
Si $n = 0$, on retourne bien la liste avec un seul chiffre nul.

Dans la suite on suppose que n est non nul : on fait au moins un tour de boucle, autrement dit $K \geq 1$.

Comme les variables L et n sont amenées à changer, on va noter L_0 et n_0 leurs valeurs initiales avant la boucle et L_k et n_k leurs valeurs à la fin du k-ème tour de boucle, pour $1 \leq k \leq K$. (Attention : L_k n'est donc pas la valeur à l'indice k de la liste L, c'est toute la liste, au tour de boucle k.)

Pour $k \in [\![1, K]\!]$, notons $\mathcal{A}(k)$ l'assertion « $n_k = n_0//2^k$ et L_k contient les k derniers chiffres de n_0 en base deux.

Montrons l'assertion $\forall k \in [\![1, K]\!], \mathcal{A}(k)$, par récurrence sur k.

1. L'assertion $\mathcal{A}(1)$ est vraie : $n_1 = n_0//2$ et $L_1 = [n_0\%2]$ est la liste contenant le dernier chiffre (celui des unités) de n_0.
2. Soit $k \in [\![1, K-1]\!]$. Supposons $\mathcal{A}(k)$. Montrons $\mathcal{A}(k+1)$. Comme $k < K$, la boucle ne termine pas à la fin du tour k, donc on sait que $n_0//2^k \neq 0$ et donc n_0 a au moins $k+1$ chiffres en base deux. Parmi ces $k+1$ derniers chiffres, les k derniers sont dans la liste L_k d'après l'hypothèse de récurrence, et le premier de ces $k+1$ derniers chiffres est aussi le dernier chiffre (ou chiffre des unités) de $n_0//2^k$. Ce chiffre des unités est $(n_0//2^k)\%2$ et d'après l'hypothèse de récurrence, ceci est égal à $n_k\%2$.
   — La première instruction de la boucle montre que L_{k+1} contient bien exactement ces $k+1$ derniers chiffres, par concaténation de $[n_k\%2]$ avec L_k.
   — La seconde instruction de la boucle dit que $n_{k+1} = n_k//2$, donc d'après l'hypothèse de récurrence, on a bien $n_{k+1} = (n_0//2^k)//2 = n_0//2^{k+1}$. (Voir remarque en début de correction.)
   Ceci conclut l'hérédité.

D'après le principe de récurrence, on a bien montré l'assertion $\forall k \in [\![1, K]\!], \mathcal{A}(k)$.

Utilisons alors l'assertion $\mathcal{A}(K)$. Par définition de K c'est-à-dire de l'arrêt de la boucle, on a $n_K = n_0//2^K = 0$ donc $n_0 < 2^K$, et aussi $n_{K-1} = n_0//2^{K-1} \neq 0$ donc $n_0 \geq 2^{K-1}$. Donc n_0 a exactement K chiffres en base deux. L'assertion $\mathcal{A}(K)$ montre que L_K contient tous les chiffres de n_0 en base deux.

Remarque. *À la place de l'assertion $\mathcal{A}(k)$, on pourrait introduire la notation ℓ_k pour l'entier dont la liste des chiffres en base deux est L_k et utiliser l'assertion $\mathcal{B}(k)$ suivante : « $n_0 = n_k 2^k + \ell_k$ ».*

Algorithme récursif

La variable d'entrée n est toujours positive par hypothèse.

```
def binaire(n: int)-> list:
  if n <= 1 : return [n]
  return binaire( n // 2 ) + [ n % 2 ]
```

Attention : ne pas remplacer la première ligne par `if n == 0 : return [0]`, sinon ça ne renvoie pas exactement ce que l'on veut.

Preuve de terminaison et correction :
Pour $n \in \mathbb{N}$, notons $\mathcal{A}(n)$ l'assertion : « l'appel à `binaire(n)` termine et renvoie la liste des chiffres de l'écriture de n en base 2 ». Montrons par récurrence forte l'assertion $\forall n \in \mathbb{N}, \mathcal{A}(n)$.

— Initialisation : $\mathcal{A}(0)$ est vraie, la fonction renvoie bien la liste contenant un seul chiffre, zéro.
— Hérédité (forte) : Soit $n \geq 1$. Supposons $\forall k < n, \mathcal{A}(k)$. Montrons $\mathcal{A}(n)$. Il y a deux cas :

1. Si $n = 1$, l'algorithme termine et renvoie bien la liste contenant un seul élément, 1, ce qu'il fallait démontrer.
2. Si $n \geq 2$, on ne rentre pas dans le « if ». Comme $n > 0$, on a $n//2 < n$, ce qui permet d'appliquer l'hypothèse de récurrence forte : l'appel à `binaire(n//2)` termine et renvoie la liste des chiffres en base deux de $n//2$. Après quoi on concatène le chiffre des unités en base 2 de n, ce qui donne bien l'écriture en base 2 de n comme rappelé plus haut.

Remarque. *Au lieu d'une récurrence forte sur n, on aurait pu faire une récurrence simple sur le nombre de chiffres en base 2 de n, mais la rédaction donnée ci-dessus semble plus simple.*

Exercice 3 : exponentiation rapide, algorithme itératif

Preuve de terminaison
idem que plus haut, il n'existe pas de suite infinie strictement décroissante d'entiers positifs.

Preuve de correction
Soit $x \in \mathbb{R}$, et $n \in \mathbb{N}$. Montrons que $f(x,n) = x^n$.

On note n_0, p_0 et r_0 les valeurs initiales des variables n, p et r.

Notons K le dernier tour de boucle effectué, avec la convention $K = 0$ s'il n'y a aucun tour de boucle.

Pour $0 \leq k \leq K$, on note n_k, p_k et r_k les valeurs des variables n, p et r à la fin du k-ème tour de boucle, et on note $\mathcal{A}(k)$ l'assertion suivante : « $p_k = x^{2^k}$, $n_k = n_0//2^k$ et $r_k = x^{n_0\%2^k}$. »

Montrons par récurrence (finie) sur k l'assertion suivante : $\forall k \in [\![0, K]\!], \mathcal{A}(k)$.

Si l'on admet momentanément ce résultat, on en déduit la correction. En effet :
- la définition de l'arrêt de la boucle donne $n_K = 0$.
- En utilisant alors la dernière assertion $\mathcal{A}(K)$, on a $n_K = n_0//2^K$. Si ce quotient entier est nul, cela veut dire que $n_0 < 2^K$ autrement dit que $n_0\%2^K = n_0$. Par la troisième partie de l'assertion $\mathcal{A}(K)$, on renvoie bien $r_K = x^{n_0\%2^K} = x^{n_0}$.

Rédigeons maintenant la récurrence.
- Initialisation : $\mathcal{A}(0)$ est vraie : $p_0 = x$, $n_0 = n_0//2^0 = n_0//1 = n_0$ et $n_0\%2^0 = n_0\%1 = 0$ donc $r_0 = x^0 = 1$.
- Soit $k \in \mathbb{N}$ tel que $k < K$. Supposons $\mathcal{A}(k)$. Montrons $\mathcal{A}(k+1)$. La boucle contient trois instructions dont une conditionnelle, qui donnent, en utilisant à chaque fois l'hypothèse de récurrence :

1. $r_{k+1} = \begin{cases} r_k = x^{n_0\%2^k} & \text{si } n_k = n_0//2^k \text{ est pair} \\ r_k p_k = x^{n_0\%2^k} x^{2^k} = x^{n_0\%2^k + 2^k} & \text{si } n_k = n_0//2^k \text{ est impair.} \end{cases}$

   On peut écrire ceci de façon inconditionnelle sous la forme :

$$r_{k+1} = x^{n_0\%2^k + 2^k \times ((n_0//2^k)\%2)}.$$

   (En effet, le terme $(n_0//2^k)\%2$ vaut 0 ou 1 suivant si $n_0//2^k$ est pair ou impair.) Montrons que ceci signifie bien que $r_{k+1} = x^{n_0\%2^{k+1}}$, autrement dit montrons que :

$$n_0\%2^{k+1} = n_0\%2^k + 2^k \times ((n_0//2^k)\%2).$$

   Preuve de l'affirmation : Écrivons la division euclidienne de $n_k = n_0//2^k$ par 2 :

$$n_0//2^k = 2 \times (n_0//2^k//2) + (n_0//2^k)\%2 = 2 \times (n_0//2^{k+1}) + (n_0//2^k)\%2$$

   En reportant cette égalité dans la division euclidienne de n_0 par 2^k on obtient

$$\begin{aligned} n_0 &= 2^k \left(2 \times (n_0//2^{k+1}) + (n_0//2^k)\%2\right) + n_0\%2^k \\ &= 2^{k+1} \left(n_0//2^{k+1}\right) + \left[2^k \times \left((n_0//2^k)\%2\right) + n_0\%2^k\right] \end{aligned}$$

   On reconnaît la division euclidienne de n_0 par 2^{k+1}, donc par unicité, le terme entre crochets doit être égal au reste :

$$2^k \times \left((n_0//2^k)\%2\right) + n_0\%2^k = n_0\%2^{k+1},$$

   ce qu'il fallait démontrer.

Finalement on a bien $r_{k+1} = x^{n_0 \% 2^{k+1}}$.

2. $p_{k+1} = (p_k)^2 = \left(x^{2^k}\right)^2 = x^{2^{k+1}}$.
3. $n_{k+1} = n_k//2 = (n_0//2^k)//2 = n_0//2^{k+1}$ (d'après la remarque en début de la correction, la remettre ici le jour J.)

Ceci conclut l'hérédité.

Remarque. *À retenir absolument : si dans une boucle on a des instructions* `p = p * p` *ou* `n = n//2`*, cela signifie que les valeurs successives de ces variables vont être de la forme* $p_0^{2^k}$ *et* $n_0//2^k$*. Dans une copie de Capes, faut sans doute néanmoins le reprouver à chaque fois. Important : pour la suite* (p_k)*, on pourrait être tenté de dire que la suite* $\ln p_k$ *est géométrique de raison* 2*, mais attention si* p_0 *est nul ou négatif.*

Remarque. *Le premier des trois points de l'hérédité est un peu plus long à rédiger. Si on regarde ce que ça signifie, c'est assez compréhensible (on regarde les premiers* $k+1$ *chiffres en base deux de* n_0*, nuls ou pas : le premier de ces chiffres, qui va être le multiplicateur devant* 2^k*, vaut* 0 *ou* 1 *suivant si* $n//2^k$ *est pair ou pas). Mais pour bien le rédiger, il faut correctement exploiter les propriétés d'unicité de la division euclidienne. En cas d'énorme blocage sur la rédaction, faire une petite note en expliquant l'idée de la preuve et admettre une partie de l'assertion.*

Remarque. *L'algorithme est donc en train de calculer l'écriture de* n *en base deux, et à la fin de l'étape* k*, la valeur* r_k *de la variable* `r` *contient* x *élevé à la puissance* $n_0\%2^k$*, qui est le nombre dont l'écriture en base est formée des* k *derniers chiffres de celle de* n*.*

Remarque. *Méthode générale : Dès qu'il faut montrer que quelque chose est égal à* $X\%Y$ *ou bien à* $X//Y$*, il faut en général utiliser l'unicité de la division euclidienne de* X *par* Y *: on fait des calculs sur* X*, on reconnaît qu'il s'agit de la division euclidienne de* X *par* Y*, puis on en déduit une identification de certains termes avec* $X//Y$ *ou* $X\%Y$*, par unicité.*

Remarque. *Rédaction alternative. On peut utiliser l'assertion* $\mathcal{B}(k)$ *suivante : « $x^{n_0} = r_k p_k^{n_k}$ » (Équivalente à* $x^{n_0} = r_k x^{2^k n_k}$*.) C'est l'approche que j'ai donnée en TD. C'est un peu plus efficace mais peut-être plus astucieux, en tout cas ça nécessite peut-être plus d'habitude. L'assertion* $\mathcal{A}(k)$ *est plus systématique et scolaire, elle contient une description de la situation globale à chaque étape, sans chercher à simplifier.*

Remarque. *Si l'on rédige à l'informaticienne, on rédigerait la correction comme ceci : «* `rp^n` *est un invariant de boucle ». Avant la boucle cet invariant vaut* x^n*, à la fin par définition* $n = 0$ *donc il vaut* r*. Pour prouver proprement l'invariant de boucle, on doit montrer que* `rp^n` *a la même valeur au début et à la fin de tout tour de boucle (en gros, on doit montrer l'hérédité d'une assertion que l'on n'a pas explicitement introduite). Pour rédiger cela, en général cela conduit à noter les valeurs en début de boucle* r'*,* p' *et* n'*, à les noter* r''*,* p'' *et* n'' *en fin de boucle, et à montrer que* $r'p'^{n'} = r''p''^{n''}$*. Ça commence à être très proche de la rédaction mathématique sauf qu'on ne dispose pas de l'indice de boucle* k *qui peut être utile. Dans une copie de concours de mathématiques, il va falloir rédiger tous les détails, autant introduire les* n_k*,* r_k *et* p_k *et faire la récurrence proprement.*

Exercice 4 : Exponentiation rapide récursive

(On rappelle que l'entrée `n` est supposée être un entier naturel.)

```
def f(x, n):
  if n == 0:
    return 1
  y = f(x, n//2)
  if n % 2 == 0:
    return y * y
  else:
    return y * y * x
```

Preuve de correction (inclut la terminaison) :

Soit $x \in \mathbb{R}$. Pour $n \in \mathbb{N}$, notons $\mathcal{A}(n)$ l'assertion « l'appel à $f(x, n)$ termine et renvoie x^n. »

Remarque. *Noter que ces assertions dépendent aussi de x même s'il n'est pas visible dans la notation. On pourrait à la rigueur utiliser une notation comme $\mathcal{A}_x(n)$ pour marquer la dépendance, mais ici le x est le même dans toute la preuve, et il n'y a pas d'ambiguïté, il vaut donc mieux ne pas alourdir les notations. Par contre je déconseille une notation comme $\mathcal{A}(x, n)$ car ça met x et n sur le même plan et ça peut provoquer des confusions.*

Montrons l'assertion $\forall n \in \mathbb{N}, \mathcal{A}(n)$, par récurrence forte sur n.

— **Initialisation** : Si l'entier en entrée est nul, on entre dans le premier `if` et la fonction termine en renvoyant $1 = x^0$. Donc $\mathcal{A}(0)$ est vraie.

— **Hérédité sous hypothèse de récurrence forte** : Soit $n \geq 1$. Supposons $\forall k < n, \mathcal{A}(k)$. Montrons $\mathcal{A}(n)$. Comme $n \geq 1$, on ne rentre pas dans le premier « if ». Comme $n \geq 1$, on a $n//2 < n$. Par hypothèse de récurrence forte, l'appel à $f(x, n//2)$ termine et renvoie $x^{n//2}$.

(À ce stade, comme il n'y a plus de boucles ou d'appels récursifs, on peut déjà conclure que l'appel à $f(x, n)$ termine, mais il reste à prouver que la valeur renvoyée est la bonne.)
Définissons $y = f(x, n//2)$ qui est donc égal à $x^{n//2}$ et écrivons la division euclidienne de n par deux :

$$n = 2 \times (n//2) + (n\%2).$$

On a donc :

$$x^n = x^{2\times(n//2)+(n\%2)} = x^{n//2} \times x^{n//2} \times x^{n\%2} = y \times y \times x^{n\%2}.$$

Autrement dit, x^n vaut $y^2 x$ si $n\%2$ est égal à 1, et sinon, il vaut y^2. C'est bien ce que renvoie la fonction avec le test conditionnel sur $n\%2$, donc $\mathcal{A}(n)$ est vraie.

D'après le principe de récurrence forte, on a bien montré : $\forall n \in \mathbb{N}$, l'appel à $f(x, n)$ termine et renvoie x^n. Comme le réel x a été fixé arbitrairement au début de la preuve, on a bien montré : « $\forall x \in \mathbb{R}, \forall n \in \mathbb{N}$, l'appel à $f(x, n)$ termine et renvoie x^n. »